

Mobile Development — Exam Revision Pack

IUGET · Computer Engineering · Level 3 · Instructor: Mr. Smith Wills · Academic Year 2025–2026

Exam format (2026): 3 hours · Coefficient 4 · Total **100 marks**.
Section A — COMPULSORY (40 marks): 5 short questions (answer all).
Section B (60 marks): answer **3 of 5** questions — expected to draw heavily on Modules 3 & 6.

Section A — Model Answers (40 guaranteed marks)

Q1. Define "Mobile Device" + FOUR examples [8 marks]

Definition: A mobile device is a small, **portable, battery-powered computing device** that can **connect wirelessly** to networks (cellular / Wi-Fi) and be operated **on the move**, typically using a touch screen and running a mobile operating system.

Four examples:

- 1. **Smartphone** — e.g. Samsung Galaxy, iPhone
- 2. **Tablet** — e.g. iPad, Samsung Galaxy Tab
- 3. **Smartwatch / wearable** — e.g. Apple Watch
- 4. **E-reader** — e.g. Amazon Kindle (*or laptop, PDA, fitness tracker*)

Q2. Mobile OS vs Desktop OS + TWO examples each [8 marks]

Note: this compares the operating systems (the software), not the hardware of the devices.

Aspect (OS behaviour)	Mobile OS	Desktop OS
Interface paradigm	Touch-first; apps run full-screen, one at a time	Windowed UI for mouse/keyboard; many overlapping windows
App installation	Apps come from a curated, sandboxed app store ; installation restricted	Software installed freely from any source
Sandboxing & permissions	Each app isolated ; must request runtime permissions (camera, location)	Apps usually run with the user's full privileges
Multitasking	Background apps suspended/limited to save power & memory	Full concurrent multitasking of many heavy apps
Power/resource management	OS aggressively manages battery & RAM (closes idle apps)	Assumes mains power; less aggressive management
File-system access	File system hidden/restricted from user and apps	Full, user-visible file system

- **Mobile OS examples:** Android, iOS
- **Desktop OS examples:** Microsoft Windows, macOS (or Linux)

Q3. Native vs Hybrid vs Web apps + ONE advantage each [9 marks]

- **Native app** — built in the platform's official language (Android → Kotlin/Java; iOS → Swift), compiled to machine code, uses native UI. *e.g. WhatsApp.*
Advantage: Best performance and full access to device hardware/APIs.
- **Hybrid app** — web code (HTML/CSS/JS) wrapped in a native shell (WebView) using frameworks like Ionic/Cordova. *e.g. older Salesforce.*
Advantage: One single codebase → cheap and fast to develop for both platforms.
- **Web app (PWA)** — runs in the browser, no app-store install needed; can work offline & send push notifications. *e.g. Twitter/X, Starbucks.*
Advantage: No installation/download required and instantly updatable across all devices.

Q4. THREE common Mobile Security Threats [8 marks] (Module 6)

- 1. **Insecure Data Storage** — sensitive data (passwords, tokens) stored in **plaintext** (SQLite, SharedPreferences, logs) can be read by attackers on a lost/rooted device.
- 2. **Insecure Communication (Man-in-the-Middle)** — on public Wi-Fi an attacker sets up a **rogue access point** to intercept unencrypted traffic and steal session data.

3. **Phishing / Smishing** — fraudulent emails or **SMS texts** trick users into giving credentials; smishing works well on mobile because URLs are hard to verify on small screens.

Other valid answers: Broken Authentication, Insufficient Cryptography, Excessive Permissions, Reverse Engineering.

Q5. App Monetization + THREE models [7 marks]

App monetization = the strategies developers use to **generate revenue** from a mobile app.

1. **In-app advertising** — earn money by displaying ads (paid per impression/click); app is free to users.
2. **In-app purchases / Freemium** — app is free, but users pay for premium features, virtual goods, or to remove ads.
3. **Subscription model** — users pay a **recurring fee** (monthly/yearly) for ongoing access. *e.g. Netflix, Spotify.*

Also valid: Paid/premium upfront download, sponsorship/affiliate marketing.

Module 3 — Mobile Networks

Section B priority

- **Mobile / cellular network** = wireless infrastructure dividing land into **cells**, each served by a **Base Transceiver Station (cell tower)**.
- **Key components:** UE (device) · SIM card · Cell Tower/Base Station · Base Station Controller (BSC) · **Mobile Switching Centre (MSC)** · Core Network · Backhaul links.

Generations of mobile networks

Gen	Era	Key features
1G	1979-1991	Analogue voice only; no SMS/data; no encryption
2G	1991-2000s	Digital voice, SMS , encryption; GSM introduced the SIM card (+GPRS/EDGE)
3G	2001-2010s	Mobile broadband (UMTS/WCDMA); web, email, video calls; min 200 Kbps
4G LTE	2010-now	Ultra-fast broadband (100 Mbps+); streaming, ride-sharing
5G	2019-now	<1 ms latency, 1M devices/km², 20 Gbps peak, network slicing
6G	2030+ (research)	THz frequencies, ~1 Tbps, native AI in the network

- **Cell types (largest → smallest):** Macrocell > Microcell > Picocell > Femtocell > Small cell (5G).
- **Three network domains:** Radio Access Network (RAN) → Transport/Backhaul → Core Network (EPC for 4G, 5GC for 5G).
- **Handover (handoff)** = seamless transfer of your connection to a stronger tower as you move, without dropping the call.
- **Cell tower parts:** Tower/mast · Antenna arrays (3 × 120° sectors) · Remote Radio Unit (RRU) · Baseband Unit (BBU) · power supply · backhaul · shelter/cabinet.
- **IMEI** = device's unique digital fingerprint; **SIM** = subscriber identity. Tower names: BTS (2G/3G), eNodeB (4G), gNodeB (5G).
- **Cameroon operators:** MTN, Orange, CAMTEL, Nexttel (Viettel). Regulator: TRB.

Module 6 — Mobile Application Security

Section B priority

Definition: the discipline of identifying, mitigating, and preventing security vulnerabilities in mobile apps and their backends so they protect user data, keep integrity, and resist attacks.

Why mobile security is uniquely challenging

Physical device loss/theft · fragmented Android ecosystem (slow patches) · sideloading · insecure public Wi-Fi · resource constraints · broad permission models · risky third-party SDKs.

Common attack vectors (7)

Threat	Prevention
Insecure Data Storage (plaintext SQLite, SharedPreferences, logs)	EncryptedSharedPreferences / iOS Keychain; SQLCipher; strip debug logs
Insecure Communication (MITM, rogue access point)	HTTPS everywhere; certificate pinning ; Network Security Config / ATS
Broken Authentication / Session Mgmt	Strong passwords; store tokens in Keystore/Keychain; short token life; MFA/biometrics
Insufficient Cryptography (MD5/SHA-1/DES, hardcoded keys)	AES-256-GCM; bcrypt/Argon2/PBKDF2; hardware key generation
Excessive Permissions / Privacy Violations	Principle of least privilege; contextual permission requests
Code Injection & Reverse Engineering	Obfuscation (ProGuard/R8); no hardcoded secrets; integrity checks
Social Engineering (Phishing / Smishing)	User education; SMS OTP with clear sender IDs; App Links/Universal Links

OWASP Mobile Top 10

M1 Improper Credential Usage · M2 Inadequate Supply-Chain Security · M3 Insecure Authentication/Authorization · M4 Insufficient Input/Output Validation · M5 Insecure Communication · M6 Inadequate Privacy Controls · M7 Insufficient Binary Protections · M8 Security Misconfiguration · M9 Insecure Data Storage · M10 Insufficient Cryptography.

Best practices & platform features

- **Secure SDLC:** threat modelling (**STRIDE**), input validation, secrets management, dependency auditing, secure API design (JWT/OAuth2, rate limiting).
- **Android:** Android Keystore (TEE), Network Security Config, Play Integrity API, SELinux.
- **iOS:** Secure Enclave, Keychain, App Transport Security (ATS), code signing.
- **Testing:** SAST (MobSF, SonarQube) · DAST (Burp Suite, OWASP ZAP, Frida) · penetration testing · dependency scanning (Snyk).

Cameroon / Africa context

Mobile Money fraud (MTN MoMo, Orange Money) — SIM-swap fraud · SMS OTP interception via SS7 weaknesses · counterfeit apps spread on WhatsApp · weak 4-digit PINs on stolen phones.

Key takeaway: Mobile security is not a final step — it is woven through every architectural decision, line of code, library choice, and backend API. Build with security as a first-class requirement from day one.